

Instance Methods (CodeGrade)

<https://github.com/learn-co-curriculum/python-p3-instance-methods><https://github.com/learn-co-curriculum/python-p3-instance-methods/issues/new>

Learning Goals

- Describe an instance method.
- Call instance methods on an object.
- Build instance methods for an object.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Initialize**: create a working copy of a class using its `__init__()` method.
- **Instance**: one specific working copy of a class. It is created when a class's `__init__()` method is called.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.
- **Attribute**: variables that belong to an object.

Introduction

We know that classes are a collection of data and functionality that act as a factory for our objects. We create those objects by calling our class just as we would a function, with closed parentheses:

```
class Dog:  
    pass
```

```
fido = Dog()  
# <__main__.Dog object at 0x1027f07f0>
```

But what can this instance of a dog stored in the local variable `fido` do? In fact, how do we even ask this object to do something?

The Behavior of Objects

Objects on their own don't do very much. In order to see the behavior of an object, we need to interact with it to elicit a response. There are certain strategies for interacting with objects that allow us to access their data and functionality from the outside.

Dot Notation

We send objects messages asking them to perform an operation or task through a syntax known as "Dot Notation".

```
class Dog:  
    pass
```

```
fido = Dog()  
# <__main__.Dog object at 0x1027f07f0>
```

```
fido.__dir__()  
# ['__module__', '__dict__', '__weakref__', '__doc__', '__repr__', '__hash__',  
# '__str__', '__getattribute__', '__setattr__', '__delattr__', '__lt__',  
# '__le__', '__eq__', '__ne__', '__gt__', '__ge__', '__init__', '__new__',  
# '__reduce_ex__', '__reduce__', '__subclasshook__', '__init_subclass__',  
# '__format__', '__sizeof__', '__dir__', '__class__']
```

When we interact with an object through dot notation, we are calling the variable or function that is defined inside of the object. We call these instance variables **attributes** and these instance functions **methods**. Unless otherwise specified, all of an object's methods are considered **instance methods**. **Instance methods** can access and modify the attributes of an instance.

In the example above, we access the `fido` instance's `__dir__()` method by separating the object, `fido` and the method, `__dir__()` by a dot (`.`).

The `__dir__()` method provides you with a list of the object's methods and attributes. You'll notice that even though we didn't provide the `Dog` class with any attributes or methods, it has many by default!

I thought of objects being like biological cells and/or individual computers on a network, only able to communicate with messages. - Alan Kay

```
"Strings are objects too.".upper()  
# 'STRINGS ARE OBJECTS TOO.'
```

Building Your Own Instance Methods

If you haven't already, fork and clone this lab to your local environment and run `pytest -x`. Follow along below to get the first few tests for the `Dog` class to pass. Once you have those passing, you will complete the remaining tasks on your own.

How do we add our own methods to our classes? In our `Dog` example, can we teach our `Dog` a new trick? Can we teach our `Dog` to bark for example?

We can. We're used to defining functions already with the `def` keyword. If we place this function definition within the body of a class, that function becomes a specific behavior of instances of that class, not a generic procedure we can just call whenever we want.

Let's create our `Dog` class in `lib/dog.py`. It's a convention among Python developers to define each class in its own file, using the class name to determine the file name.

In the `Dog` class, let's define our `bark()` instance method:

```
class Dog:
    # Class body

    # Instance Method Definition
    def bark(self):
        print("Woof!")
```

Note the use of the `self` keyword in our instance method definition above. We will explore the concept of `self` in depth in the next module- just know for now that it allows methods to access the other methods and attributes. `self` is **required** in instance method definitions, though you don't actually need to pass it in when you call the method.

With this code in place, if you run the tests you should now be passing the first three tests.

Now let's create an instance of `Dog` and verify that our `Dog` object knows how to bark:

```
class Dog:
    def bark(self):
        print("Woof!")

fido = Dog()
fido.bark()
# Woof!
```

If you execute this code in the terminal by running `python lib/dog.py` , you should see "Woof!" written out.

By defining `bark()` within the `Dog` class, `bark` becomes a method of all instances of Dogs. Let's create a second instance of `Dog` , `snoopy` , and verify that snoopy also knows how to bark by running `python lib/dog.py` again.

```
class Dog:
    def bark(self):
        print("Woof!")
```

```
fido = Dog()
fido.bark()
# Woof!
```

```
snoopy = Dog()
snoopy.bark()
# Woof!
```

Objects can only do what we teach them to do via the code we write and the methods we define. For example, currently, Dogs do not know how to sit.

```
class Dog:
    def bark(self):
        print("Woof!")
```

```
fido = Dog()
fido.bark()
# Woof!
```

```
fido.sit()
# AttributeError: 'Dog' object has no attribute 'sit'
```

In the same manner, instance methods, the methods that belong to particular instances of particular classes, cannot be used globally like procedural methods. They cannot be called without an instance.

```
class Dog:
    def bark(self):
        print("Woof!")
```

```
fido = Dog()
fido.bark()
# Woof!
```

```
# Let's try just calling bark without fido
bark()
# NameError: name 'bark' is not defined
```

Instructions

Complete the following tasks to get the rest of the tests passing:

1. Add an instance method `sit()` to your `Dog` class that will print "The dog is sitting."
 2. Define a `Person` class in `lib/person.py`
 3. Add an instance method `talk()` to your `Person` class that will print "Hello World!"
 4. Add an instance method `walk()` to your `Person` class that will print "The person is walking."
-

Conclusion

With all tests passing, you have successfully written multiple instance methods and *two* different classes!

The ability to define methods and behaviors in our classes for our instances makes Python classes behave not just as factories, capable of instantiating new individual instances, but also as a blueprint, defining what those instances can do.

Resources

- [Python documentation](https://docs.python.org/3/)  [\(https://docs.python.org/3/\)](https://docs.python.org/3/)
- [Instance method in Python](https://www.geeksforgeeks.org/instance-method-in-python/)  [\(https://www.geeksforgeeks.org/instance-method-in-python/\)](https://www.geeksforgeeks.org/instance-method-in-python/)
- [Method Objects](https://docs.python.org/3/tutorial/classes.html#method-objects)  [\(https://docs.python.org/3/tutorial/classes.html#method-objects\)](https://docs.python.org/3/tutorial/classes.html#method-objects)

This tool needs to be loaded in a new browser window

Load Instance Methods (CodeGrade) in a new window